

Doc No: P0034
Date: 2015-09-25
Reply to: Bill Seymour <stdbill.h@pobox.com>

Civil Time for the Standard Library

Bill Seymour
2015-09-25

Abstract:

This paper asks, what's your interest in adding to the standard library classes that support civil time beyond what `<ctime>` provides?

The [Boost.Date_Time](#) library would certainly do the trick; but it's really huge; and this writer isn't aware of any interest in accepting it into the standard library.

This paper suggests a simpler design that just does the basics. In particular, it assumes that we won't need to deal with any pre-Gregorian-calendar dates, or even any `time_t`-overflow dates; and although we might want sub-second *resolution* (for example, to handle SQL `TIMESTAMPS`), we won't really need the kind of *accuracy* that requires handling leap seconds any better than `time_t` does.

If something like `Boost.Date_Time` does make its way into the standard library, this proposal will be withdrawn.

Issue: Although this paper shows `operator ""` literals for several types because they might be convenient for users, we might not want them anyway since they wouldn't be `constexpr` (unless we require the compiler to have in it a parser for all the various [ISO 8601](#) formats plus the additional forms that this paper requires).

Overview:

`class duration;`

This is a signed amount of time with microsecond resolution. It supports all the usual integer arithmetic. Other `duration`s can be added to it or subtracted from it; it can be multiplied and divided by `long` `long`s; and the quotient of two `duration`s is a `long` `long`.

`class time_of_day;`

This is an unsigned amount of time after midnight. It's like a `duration`, but restricted to the half-open interval, [midnight, midnight +1 day). Users may select whether it saturates or modwraps. It does not understand leap seconds, and cannot since it's not associated with any particular day. A pure-number multiplier or divisor, and the quotient of two `time_of_day`s, is an unsigned `long`, not a `long` `long`.

Objects of the `duration` and `time_of_day` types can freely interact arithmetically.

`class month_count;`

This is a signed number of months. Objects of this type cannot interact arithmetically with `duration`s or `time_of_day`s; but `month_count`s can be added and subtracted; they can be multiplied and divided by `int`s; and the quotient of two `month_count`s is an `int`.

`class datetime;`

This is a point in time in a particular time zone. `duration`s, `time_of_day`s, and `month_count`s can be added to it and subtracted from it; but this class provides no multiplicative arithmetic. The difference between two `datetime`s is a `duration`. `datetime`s can be explicitly converted to and from `time_t` and `struct tm`.

`class timezone;`

This class has member functions that return `duration`s that are offsets from UTC, `bool`s that indicate whether daylight saving time is in effect, and `strings` that contain the customary abbreviations for the time zone name (e.g., "CST"). The return values can reflect the local wall clock time or a particular `datetime`.

As specified in [ISO 8601](#), Eastern-hemisphere time zones have positive offsets.

A `timezone` can be constructed from a POSIX string like `"CST6CDT"`, an ISO 8601 time zone specifier, or optionally (if the implementation supports the Olson zoneinfo database), a name like `"America/Chicago"`. There's also a default constructor that constructs the local time zone; but the local time zone also exists as a constant at namespace scope, so the user never needs to explicitly construct it.

There are also some obvious `constexpr` values (e.g., `secs_per_day`) at namespace scope. They're intended to help with avoiding magic numbers.

I/O is coming Real Soon Now, probably involving extensions to the existing `time_get` and `time_put` facets.

The library provides no `clock()`-related interface: the `<chrono>` library is a better choice for that.

Synopsis:

For brevity herein, constructors that take string arguments and the operator `" "` literals are shown only in `const char*` form. You may assume that all the standard character and string types are required as well.

```
namespace std {
namespace experimental {
namespace civiltime {

class duration;
class time_of_day;
class month_count;
class datetime;
class timezone;

//
// Most of the types in the library will have microsecond resolution
// so that they can map to third-party types that do (e.g., various
// SQL types that hold dates and times).
//
typedef long long microsecs;

//
// Some interesting constants:
//

constexpr int months_per_year = 12;

constexpr long hours_per_day = 24L;
constexpr long mins_per_hour = 60L;
constexpr long secs_per_min  = 60L;

constexpr long mins_per_day  = hours_per_day * mins_per_hour;
constexpr long secs_per_day  = mins_per_day  * secs_per_min;
constexpr long secs_per_hour = mins_per_hour * secs_per_min;

constexpr long usecs_per_sec = 1000000L;

//
// Note that the only requirements on the time_t type are that it be
// an arithmetic type and that it "holds a time", whatever that means.
//
extern const long usecs_per_time_t_tick;

//
// A duration is a signed amount of time with microsecond resolution.
// It can be constructed from, and broken down into, days, hours,
// minutes, seconds, and microseconds, but not into years or months.
//
class duration
{
public:
    static constexpr duration one_day;
    static constexpr duration one_hour;
    static constexpr duration one_minute;
    static constexpr duration one_second;
    static constexpr duration zero;

    explicit constexpr duration(microsecs = 0LL) noexcept;

    duration(int hours,
             int minutes,
             int seconds = 0,
```

```

        long microseconds = 0L) noexcept;

duration(int days,
         int hours,
         int minutes,
         int seconds,
         long microseconds) noexcept;

//
// Construct from an optional sign ('+', '-', U+2212) followed by:
// - an ISO 8601 duration without years or months
// - an ISO 8601 time interval
// - a string like "hh:mm:ss"
// - a string like "dd;hh:mm:ss" (the day separator is a semicolon,
//   not a colon)
//
// In all forms, the seconds may contain a decimal point
// and up to six decimal digits; and less significant fields
// (along with their leading separators) are optional, except
// that the semicolon is always required in the fourth form,
// even to specify just a number of days.
//
explicit duration(const char*);

//
// We'd like to construct a duration from a time_t; but we can't
// write such a constructor because a time_t might be just a
// typedef for a long long; and if it is, the unit is probably
// seconds, not microseconds. Here's a convenience function
// that lets us write, e.g., duration(from_tt(my_time_t));
//
static microsecs from_tt(std::time_t) noexcept;

//
// The above isn't a problem for a struct tm; but we'll provide
// the same mechanism since that seems to support the principle
// of least astonishment (because tm is another <ctime> type
// and so could reasonably be handled in some parallel way).
//
// The following function has the side effect of calling std::mktime(),
// which is why the argument is a pointer to non-const tm.
//
static microsecs from_tm(std::tm*);
// and for the user's convenience:
static microsecs from_tm(std::tm& tmref) { return from_tm(&tmref); }

//
// The author imagines that the internal representation could be
// just a long long or other fundamental type, so there's no need
// for move semantics or swap. Implementors could certainly add
// them; but they shouldn't be required.
//

duration(const duration&) = default;
duration& operator=(const duration&) = default;

~duration() = default;

//
// Observers:
//

microsecs total_microsecs() const noexcept;

struct breakdown
{
    int days, hours, minutes, seconds;
    long microseconds;
};
explicit operator breakdown() const noexcept;

//
// Unary + and - return copies:
//
duration operator+() const noexcept;
duration operator-() const noexcept;

//
// The prefix and postfix increment and decrement operators

```

```

// add and subtract whole seconds, not microseconds:
//
duration& operator++() noexcept;
duration& operator--() noexcept;
duration operator++(int) noexcept;
duration operator--(int) noexcept;

//
// Other durations and time_of_day can be added and subtracted:
//
duration& operator+=(const duration&) noexcept;
duration& operator-=(const duration&) noexcept;
duration& operator+=(const time_of_day&) noexcept;
duration& operator-=(const time_of_day&) noexcept;

//
// durations can be multiplied and divided by pure numbers:
//
duration& operator*=(long long) noexcept;
duration& operator/=(long long) noexcept;

//
// The usual integer remainder operations:
//
duration& operator%=(long long) noexcept;
duration& operator%=(const duration&) noexcept;
duration& operator%=(const time_of_day&) noexcept;

//
// Comparisons:
//

bool operator==(const duration&) const noexcept;
bool operator!=(const duration&) const noexcept;
bool operator< (const duration&) const noexcept;
bool operator> (const duration&) const noexcept;
bool operator<=(const duration&) const noexcept;
bool operator>=(const duration&) const noexcept;

bool operator==(const time_of_day&) const noexcept;
bool operator!=(const time_of_day&) const noexcept;
bool operator< (const time_of_day&) const noexcept;
bool operator> (const time_of_day&) const noexcept;
bool operator<=(const time_of_day&) const noexcept;
bool operator>=(const time_of_day&) const noexcept;
};

duration operator+(const duration&, const duration&) noexcept;
duration operator-(const duration&, const duration&) noexcept;

duration operator+(const duration&, const time_of_day&) noexcept;
duration operator-(const duration&, const time_of_day&) noexcept;

duration operator*(const duration&, long long) noexcept;
duration operator*(long long, const duration&) noexcept;
duration operator/(const duration&, long long) noexcept;

long long operator/(const duration&, const duration&) noexcept;
long long operator/(const duration&, const time_of_day&) noexcept;

duration operator%(const duration&, long long) noexcept;
duration operator%(const duration&, const duration&) noexcept;
duration operator%(const duration&, const time_of_day&) noexcept;

//
// We'll also provide std::div()-like operations:
//
std::pair<duration, duration> div(const duration&, long long) noexcept;
std::pair<long long, duration> div(const duration&, const duration&) noexcept;
std::pair<long long, duration> div(const duration&, const time_of_day&) noexcept;

//
// duration literals...same requirements as
// construction from strings; NB: not constexpr:
//
duration operator "" dur(const char*, size_t);

//
// A time_of_day is an unsigned amount of time after midnight.

```

```

// It does not understand leap seconds since it's not associated
// with any particular day. Values will always be in the half-open
// interval, [midnight, midnight +1 day). Users may select whether
// arithmetic saturates or modwraps. The default is to modwrap.
//
class time_of_day
{
public:
    explicit constexpr time_of_day(bool saturate = false); // midnight

    explicit time_of_day(unsigned long tot_seconds, bool saturate = false);

    time_of_day(unsigned int  hour,
                unsigned int  minute,
                unsigned int  second = 0U,
                unsigned long microsecond = 0UL,
                bool saturate = false);

    //
    // Construct from strings as for duration, but only:
    //   - an ISO 8601 duration without years, months or days
    //   - a string like "hh:mm:ss"
    // with the optional decimal point for non-zero microseconds.
    // A leading sign is not allowed.
    //
    explicit time_of_day(const char*);

    time_of_day(const time_of_day&) = default;
    time_of_day& operator=(const time_of_day&) = default;

    ~time_of_day() = default;

    //
    // Get/set saturate/modwrap behavior:
    //
    bool get_saturate() const noexcept;
    void set_saturate(bool = true) noexcept;

    //
    // A time_of_day can be cast to a duration.
    // You get a copy, not a reference to *this,
    // since duration values aren't constrained.
    //
    explicit operator duration() const noexcept;

    microsecs total_microsecs() const noexcept;

    struct breakdown
    {
        unsigned int hour, minute, second;
        unsigned long microsecond;
    };
    explicit operator breakdown() const noexcept;

    //
    // Since *this is conceptually unsigned, unary + and -
    // don't seem particularly useful; but they're there
    // anyway just for the sake of least astonishment.
    //
    time_of_day operator+() const noexcept; // gets a copy
    time_of_day operator-() const noexcept; // saturates or modwraps

    time_of_day& operator++() noexcept;
    time_of_day& operator--() noexcept;

    time_of_day operator++(int) noexcept;
    time_of_day operator--(int) noexcept;

    time_of_day& operator+=(const time_of_day&) noexcept;
    time_of_day& operator-=(const time_of_day&) noexcept;

    time_of_day& operator+=(const duration&) noexcept;
    time_of_day& operator-=(const duration&) noexcept;

    time_of_day& operator*=(unsigned long) noexcept;
    time_of_day& operator/=(unsigned long) noexcept;

    time_of_day& operator%=(unsigned long) noexcept;
    time_of_day& operator%=(const time_of_day&) noexcept;

```

```

time_of_day& operator%=(const duration&) noexcept;

bool operator==(const time_of_day&) const noexcept;
bool operator!=(const time_of_day&) const noexcept;
bool operator< (const time_of_day&) const noexcept;
bool operator> (const time_of_day&) const noexcept;
bool operator<=(const time_of_day&) const noexcept;
bool operator>=(const time_of_day&) const noexcept;

bool operator==(const duration&) const noexcept;
bool operator!=(const duration&) const noexcept;
bool operator< (const duration&) const noexcept;
bool operator> (const duration&) const noexcept;
bool operator<=(const duration&) const noexcept;
bool operator>=(const duration&) const noexcept;
};

time_of_day operator+(const time_of_day&, const time_of_day&) noexcept;
time_of_day operator-(const time_of_day&, const time_of_day&) noexcept;

time_of_day operator+(const time_of_day&, const duration&) noexcept;
time_of_day operator-(const time_of_day&, const duration&) noexcept;

time_of_day operator*(const time_of_day&, unsigned long) noexcept;
time_of_day operator*(unsigned long, const time_of_day&) noexcept;
time_of_day operator/(const time_of_day&, unsigned long) noexcept;

unsigned long operator/(const time_of_day&, const time_of_day&) noexcept;
/*signed*/ long operator/(const time_of_day&, const duration&) noexcept;

time_of_day operator%(const time_of_day&, unsigned long) noexcept;
time_of_day operator%(const time_of_day&, const time_of_day&) noexcept;
time_of_day operator%(const time_of_day&, const duration&) noexcept;

//
// The div()-like functions need to allow the user to specify
// the saturate/modwrap behavior ...
//
std::pair<time_of_day, time_of_day>
div(const time_of_day&, unsigned long, bool) noexcept;
std::pair<long, time_of_day>
div(const time_of_day&, const time_of_day&, bool) noexcept;
std::pair<long, time_of_day>
div(const time_of_day&, const duration&, bool) noexcept;
//
// or just default to the first argument's behavior:
//
std::pair<time_of_day, time_of_day>
div(const time_of_day&, unsigned long) noexcept;
std::pair<long, time_of_day>
div(const time_of_day&, const time_of_day&) noexcept;
std::pair<long, time_of_day>
div(const time_of_day&, const duration&) noexcept;

//
// time_of_day literals:
//
time_of_day operator "" tod(const char*, size_t);

//
// A month_count is a signed amount of time with month resolution.
// A month_count cannot interact arithmetically with either duration
// or time_of_day.
//
class month_count
{
public:
    static constexpr month_count one_year;
    static constexpr month_count one_month;
    static constexpr month_count zero;

    explicit constexpr month_count(int tot_months = 0) noexcept;
    constexpr month_count(int years, int months) noexcept;

    //
    // Construct from:
    // - an ISO 8601 duration with only 'Y' and 'M' suffixes
    // - a string containing years, a hyphen, and months
    // - a string containing total months

```

```

// with an optional leading sign
//
explicit month_count(const char*);

month_count(const month_count&) = default;
month_count& operator=(const month_count&) = default;

~month_count() = default;

int years() const noexcept;
int months() const noexcept;
int total_months() const noexcept;

typedef std::pair<int,int> breakdown;
explicit operator breakdown() const noexcept;

month_count operator+() noexcept;
month_count operator-() noexcept;

month_count& operator++() noexcept;
month_count& operator--() noexcept;

month_count operator++(int) noexcept;
month_count operator--(int) noexcept;

month_count& operator+=(month_count) noexcept;
month_count& operator-=(month_count) noexcept;

month_count& operator*=(int) noexcept;
month_count& operator/=(int) noexcept;
month_count& operator%=(int) noexcept;
month_count& operator%=(month_count) noexcept;

bool operator==(month_count) const noexcept;
bool operator!=(month_count) const noexcept;
bool operator< (month_count) const noexcept;
bool operator> (month_count) const noexcept;
bool operator<=(month_count) const noexcept;
bool operator>=(month_count) const noexcept;
};

month_count operator+(const month_count&, month_count) noexcept;
month_count operator-(const month_count&, month_count) noexcept;

month_count operator*(const month_count&, int) noexcept;
month_count operator*(int, const month_count&) noexcept;
month_count operator/(const month_count&, int) noexcept;

int operator/(month_count, month_count) noexcept;

month_count operator%(const month_count&, int) noexcept;
month_count operator%(const month_count&, month_count) noexcept;

std::pair<month_count, month_count> div(month_count, int) noexcept;
std::pair<int, month_count> div(month_count, month_count) noexcept;

month_count operator "" mc(const char*, size_t);

//
// A datetime is a point in time in some time zone.
//
class datetime
{
public:
    datetime(); // the current wall clock time in the local time zone

    explicit datetime(const timezone&); // current wall clock time

    explicit datetime(const duration&); // in the local time zone
    explicit datetime(std::time_t);
    explicit datetime(std::tm*);
    explicit datetime(std::tm&);

    datetime(const duration&, const timezone&);
    datetime(std::time_t, const timezone&);
    datetime(std::tm*, const timezone&);
    datetime(std::tm&, const timezone&);

    //

```

```

// Construct from an ISO 8601 date/time representation
// (the 'T' separator may not be replaced by a space):
//
explicit datetime(const char*);

//
// A timezone will require move semantics and swap,
// so a datetime will, too.
//
explicit datetime(timezone&&);
datetime(const duration&, timezone&&);
datetime(std::time_t, timezone&&);
datetime(std::tm*, timezone&&);
datetime(std::tm&, timezone&&);

datetime(const datetime&) = default;
datetime& operator=(const datetime&) = default;

datetime(datetime&&) = default;
datetime& operator=(datetime&&) = default;

~datetime() = default;

void swap(datetime&) noexcept;

const timezone& get_timezone() const noexcept;

explicit operator std::time_t() const noexcept;

struct breakdown
{
    int year; // /not/ -1900 like std::tm
    int mon; // 1-based (not like std::tm)
    int mday; // 1-based (like std::tm)

    int wday; // 0 == Sunday
    int yday; // 0-based

    int hour; // 0 to 23
    int min; // 0 to 59
    int sec; // 0 to 60 (allow leap sec.)
    long usec; // 0 to 999,999

    int isdst; // same as std::tm

    duration utc_offset;

    explicit operator std::tm() const noexcept;
};
breakdown localtime() const;
breakdown gmtime() const;

//
// We'll provide mktime() functionality, first adjusting
// unreasonable values just like std::mktime() does:
//
datetime mktime(const breakdown&) noexcept; // *this' time zone
static datetime mktime(const breakdown&, const timezone&) noexcept;
static datetime mktime(const breakdown&, timezone&&) noexcept;

//
// The wall-clock time that would correspond to *this
// in the specified time zone:
//
datetime this_time_in(const timezone&) const;
datetime this_time_in(timezone&&) const;

//
// Assign a different time zone, optionally
// adjusting *this' internal value to match.
//
void set_timezone(const timezone&, bool adjust_val = false);
void set_timezone(timezone&&, bool adjust_val = false);

datetime& operator+=(const duration&) noexcept;
datetime& operator-=(const duration&) noexcept;

datetime& operator+=(const time_of_day&) noexcept;
datetime& operator-=(const time_of_day&) noexcept;

```



```

    datetime& operator+=(month_count);
    datetime& operator-=(month_count);

    bool operator==(const datetime&) const noexcept;
    bool operator!=(const datetime&) const noexcept;
    bool operator< (const datetime&) const noexcept;
    bool operator> (const datetime&) const noexcept;
    bool operator<=(const datetime&) const noexcept;
    bool operator>=(const datetime&) const noexcept;
};

void swap(datetime&, datetime&) noexcept;

datetime operator+(const datetime&, const duration&) noexcept;
datetime operator+(const duration&, const datetime&) noexcept;
datetime operator-(const datetime&, const duration&) noexcept;

datetime operator+(const datetime&, const time_of_day&) noexcept;
datetime operator+(const time_of_day&, const datetime&) noexcept;
datetime operator-(const datetime&, const time_of_day&) noexcept;

datetime operator+(const datetime&, month_count);
datetime operator+(month_count, const datetime&);
datetime operator-(const datetime&, month_count);

datetime operator "" dt(const char*, size_t);

//
// A time zone is an offset from UTC.
//
class timezone
{
public:
    timezone(); // the local time zone

    //
    // Construct the named time zone.  POSIX strings like "CST6CDT"
    // are always supported.  If you're running on a box that has
    // the Olson zoneinfo database, names like "America/Chicago"
    // may be specified as well.  An ISO 8601 time zone specifier
    // may also be used, but if you construct a timezone from such
    // a string, the timezone won't have any daylight-saving-time
    // rules associated with it.
    //
    explicit timezone(const char*);

    timezone(const timezone&) = default;
    timezone& operator=(const timezone&) = default;

    timezone(timezone&&) = default;
    timezone& operator=(timezone&&) = default;

    ~timezone() = default;

    void swap(timezone&) noexcept;

    const std::string& name() const noexcept;

    duration utc_offset() const; // at local wall clock time
    bool is_dst() const;        // "
    std::string abbrev() const; // "

    duration utc_offset(const datetime&) const; // at specified time
    bool is_dst(const datetime&) const;        // "
    std::string abbrev(const datetime&) const; // "
};

void swap(timezone&, timezone&) noexcept;

} // namespace civiltime
} // namespace experimental
} // namespace std

```